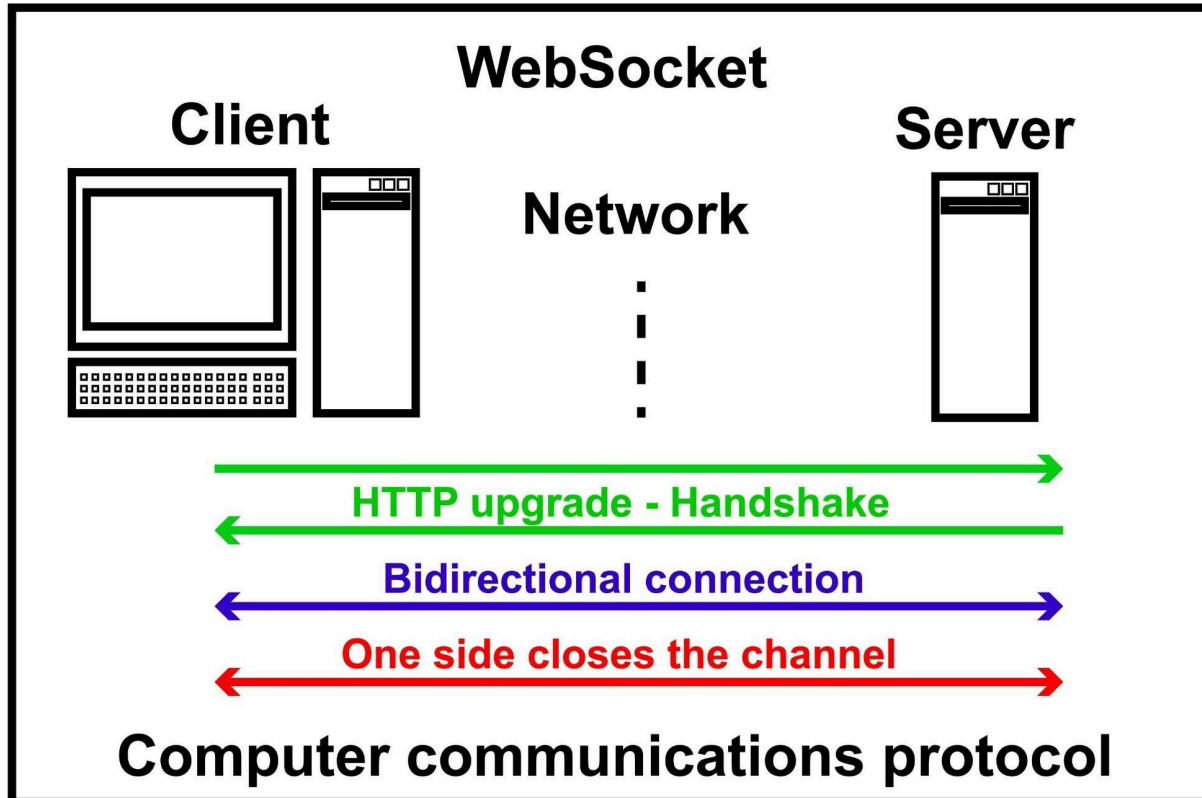

1.) Interacción con Servicios Web y APIs

La mayoría de las aplicaciones modernas necesitan comunicarse con el mundo exterior. Esto se hace a través de **APIs (Interfaces de Programación de Aplicaciones)**.

1-1. Introducción a las APIs REST

Una **API REST (Representational State Transfer)** es un conjunto de reglas que permite a las aplicaciones comunicarse a través del protocolo **HTTP**. Es el estándar *de facto* para la comunicación entre *clientes* (como nuestra aplicación Android) y *servidores*.

- **Importancia para móviles:** Permite desacoplar la lógica de negocio (en el servidor) de la interfaz de usuario (en el móvil), haciendo la aplicación más ligera, escalable y con datos siempre actualizados.
- **Principios Clave:**
 - **Cliente-Servidor:** Separación de responsabilidades.
 - **Sin Estado (Stateless):** Cada solicitud del cliente al servidor contiene toda la información necesaria para que el servidor la entienda, sin depender de sesiones anteriores.
 - **Operaciones (Verbos HTTP):** Se utilizan verbos estándar para realizar acciones sobre recursos:
 - **GET:** Recuperar datos.
 - **POST:** Crear un nuevo recurso.
 - **PUT/PATCH:** Actualizar un recurso.
 - **DELETE:** Eliminar un recurso.



Shutterstock

[Explorar](#)

1-2. Llamadas de red con Retrofit

Retrofit es la librería cliente HTTP más popular en el ecosistema Android con Java. Simplifica drásticamente las llamadas de red al convertirlas en **interfaces Java declarativas**.

- **Ventajas:** Elimina la necesidad de escribir código repetitivo para manejar conexiones, *streams* de datos y *parsing* de JSON manualmente.
- **Funcionamiento:**
 1. Definimos una **interfaz Java** con métodos que representan las llamadas a la API (por ejemplo, @GET("users")).
 2. Retrofit genera automáticamente el código de implementación que hace la solicitud HTTP real.

Java

```
public interface ApiService {
    @GET("users/{id}") // GET request to /users/{id}
    Call<User> getUser(@Path("id") int userId);

    @POST("users") // POST request to /users
    Call<User> createUser(@Body User user);
}
```

1-3. Parsing de datos JSON

El formato más común para intercambiar datos en las APIs REST es **JSON (JavaScript Object Notation)**.

- **Necesidad de Parsing:** El servidor envía una cadena de texto JSON; necesitamos convertirla en **objetos Java** (User, Product, etc.) para poder trabajar con ella en la aplicación.
- **Gson:** Es la librería de Google más utilizada para esta tarea. Actúa como un **Serializador/Deserializador** (un **convertidor**).
 - **Deserialización:** JSON (cadena) $\rightarrow$ Objeto Java. (Lo más común al recibir datos).
 - **Serialización:** Objeto Java $\rightarrow$ JSON (cadena). (Para enviar datos en un POST/PUT).

- **Integración con Retrofit:** Retrofit utiliza internamente **Gson** (o librerías similares) a través de un **ConverterFactory** para manejar el *parsing* de forma automática en base al tipo de retorno (Call<User>).

1-4. Autenticación y seguridad

Las APIs suelen requerir **Autenticación** para saber *quién* está haciendo la solicitud.

- **Tokens de Autenticación:** El método más seguro y común en REST.
 1. El cliente envía credenciales (usuario/contraseña) al *endpoint* de *login* (POST /login).
 2. El servidor verifica y, si son correctas, devuelve un **Token** (p. ej., un **JWT - JSON Web Token**).
 3. El cliente almacena este token de forma segura (p. ej., en SharedPreferences).
 4. Para todas las solicitudes futuras, el cliente incluye el token en el **Encabezado HTTP Authorization**, típicamente como **Bearer Token**.
 - **Implementación con Retrofit:** Se utiliza un **Interceptor** de **OkHttp** (la librería de red subyacente de Retrofit) para inyectar automáticamente el encabezado de autenticación en cada solicitud.
-

2.) Concurrencia e Hilos de Ejecución

La **concurrencia** es crucial en Android para mantener la aplicación fluida y receptiva.

2-1. Conceptos de hilos y procesos

- **Proceso:** Una instancia de un programa en ejecución. En Android, cada aplicación se ejecuta generalmente en su **propio proceso** aislado (una máquina virtual **ART**).
- **Hilos (Threads):** Secuencias de ejecución **dentro de un proceso**. Un proceso puede tener múltiples hilos ejecutándose al mismo tiempo (concurrentemente).
- **Hilo Principal (UI Thread):**
 - **Propósito:** Es el único hilo responsable de manejar la interfaz de usuario (**UI**), eventos de entrada (tocar, deslizar) y *lifecycle* de los componentes de Android.
 - **Regla de Oro: NUNCA** se deben realizar operaciones largas o de bloqueo (como llamadas de red, acceso a base de datos, o cálculos pesados) en el Hilo Principal, ya que lo **bloquearían**, causando que la aplicación se "congele" (**ANR - Application Not Responding**).
- **Hilos de Trabajo (Worker Threads/Background Threads):**
 - **Propósito:** Realizar tareas pesadas o de larga duración sin interferir con la UI.

2-2. Manejo de tareas asíncronas

Históricamente, **AsyncTask** fue la forma más sencilla de ejecutar tareas en segundo plano y luego actualizar la UI.

- **Uso (en desuso, pero conceptualmente importante):**
 1. **doInBackground():** Código que se ejecuta en un hilo de trabajo.
 2. **onPostExecute():** Código que se ejecuta *después* de `doInBackground()` y *vuelve automáticamente* al Hilo Principal, permitiendo actualizar la UI de forma segura.
- **Nota:** Aunque Google recomienda usar soluciones más modernas (como *Coroutines* en Kotlin o **ExecutorService** en Java/Android), **AsyncTask** sigue siendo un concepto fundamental de **programación asíncrona** en Android.

2-3. Programación con ExecutorService

ExecutorService es el método avanzado y recomendado en Java para manejar la concurrencia, proporcionando una manera de administrar un *pool* de hilos.

- **Pool de Hilos:** En lugar de crear un hilo nuevo para cada tarea, el *ExecutorService* reutiliza un conjunto limitado de hilos, lo que reduce la sobrecarga de creación y destrucción de hilos.
- **Tipos de Executors Comunes:**
 - **Executors.newFixedThreadPool(int n):** Crea un *pool* con un número fijo de hilos. Si hay más tareas, esperan.
 - **Executors.newCachedThreadPool():** Crea un *pool* que crea hilos nuevos si es necesario, pero reutiliza los inactivos.
- **Ejecución de Tareas:** Simplemente se le da un objeto **Runnable** o **Callable** (para resultados) al *ExecutorService* para que lo ejecute.

Java

```
ExecutorService executor = Executors.newFixedThreadPool(2);
```

```
executor.execute(new Runnable() {  
    @Override  
    public void run() {  
        // Tarea pesada en segundo plano  
    }  
});
```

```
// Para tareas que afectan la UI, se usa un Handler  
// o la función Activity.runOnUiThread(Runnable action)
```

2-4. Manejo de errores

El manejo de excepciones es vital en hilos de trabajo. Una excepción no capturada en un hilo de trabajo puede, en algunos casos, *crashear* la aplicación.

- **Estrategia:** Siempre se debe envolver el código potencialmente problemático dentro del hilo de trabajo en un bloque **try-catch**.

Java

```
executor.execute(new Runnable() {  
    @Override  
    public void run() {  
        try {  
            // Operación que puede fallar (p. ej. llamada a API)  
            // ...  
        } catch (IOException e) {  
            // Capturar la excepción  
            e.printStackTrace();  
            // Notificar al Hilo Principal (p. ej. usando un Handler)  
        }  
    }  
});
```

3.) Arquitectura y Patrones de Diseño 🏗️

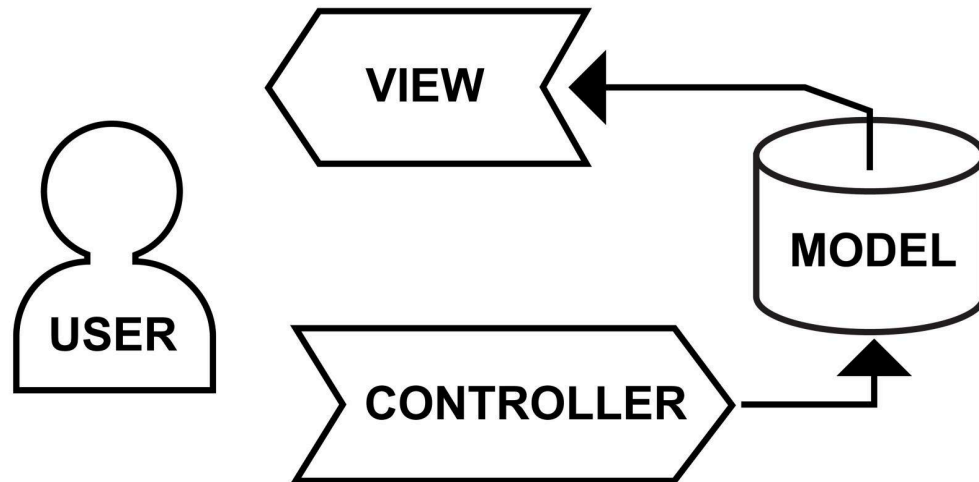
Los patrones de diseño son esenciales para crear código mantenible, *testable* y escalable.

3-1. Introducción al patrón M.V.V.M.

MVVM (Model-View-ViewModel) es el patrón de arquitectura recomendado por Google para el desarrollo moderno de Android. Su objetivo principal es la **Separación de Intereses (Separation of Concerns)**.

- **Componentes:**

- **View (Vista):** La capa de UI (actividades, fragmentos). Solo **muestra** los datos y **notifica** los eventos del usuario (clicks, etc.). **No contiene lógica de negocio.**
- **Model (Modelo):** La capa de datos. Es responsable de la lógica de negocio y el acceso a datos (APIs, base de datos).
- **ViewModel:** Un mediador entre la **View** y el **Model**.
 - Contiene la lógica de **preparación de datos para la UI.**
 - **Sobrevive a los cambios de configuración** (p. ej., rotaciones de pantalla), lo que le permite mantener los datos de la UI en memoria.



MODEL - VIEW - CONTROLLER PATTERN

Shutterstock

3-2. Componentes de la arquitectura de Android (AAC)

Los **Android Architecture Components (AAC)** son la implementación de Google de los principios MVVM.

- **ViewModel:** El componente clave para el MVVM. Almacena y gestiona los datos relacionados con la UI de forma que sobreviven a los cambios de configuración.
- **LiveData:** Un contenedor de datos **observable** que es **consciente del lifecycle** (sabe si la *View* está activa o pausada).
 - **Observación:** La **View (Activity/Fragment)** se *suscribe* a LiveData. Cuando los datos en el LiveData cambian (actualizados por el **ViewModel**), el observador solo recibe la actualización si está en un estado activo (*started* o *resumed*), eliminando el riesgo de fugas de memoria y *crashes* innecesarios.

3-3. Inyección de dependencias (DI)

La **Inyección de Dependencias (DI)** es una técnica para gestionar las dependencias de los objetos (las cosas que necesita una clase para funcionar, p. ej., una clase UserRepository necesita un ApiService).

- **Principio:** En lugar de que una clase cree sus propias dependencias, se las **inyectan** (se las pasan) desde una fuente externa.
- **Beneficios:**
 - **Código más testeable:** Podemos inyectar *Mocks* o implementaciones de prueba.
 - **Reutilización y desacoplamiento:** Las clases no están rígidamente acopladas a la implementación concreta de sus dependencias.
- **Librerías:** **Dagger** (más verboso y complejo) o su alternativa simplificada, **Hilt** (basado en Dagger, más fácil de usar en Android), son las herramientas estándar en el ecosistema, incluso en proyectos Java/Kotlin mixtos.

3-4. Patrones de repositorio

El **Patrón de Repositorio** es parte esencial del Modelo en MVVM.

- **Propósito:** Actúa como una fuente de verdad **única y centralizada** para el acceso a datos. La **ViewModel** solo se comunica con el **Repositorio**, y *nunca* directamente con las fuentes de datos (API, Base de datos local, *SharedPreferences*).
- **Ventajas:**
 - **Abstracción:** El **ViewModel** no necesita saber *de dónde* vienen los datos.
 - **Lógica de caché:** El Repositorio es el lugar ideal para implementar la lógica de si los datos deben cargarse desde la caché local o hacer una llamada de red.

Flujo de datos MVVM + Repositorio:

$$\text{View} \rightarrow \text{LiveData} \rightarrow \text{ViewModel} \rightarrow \text{Repository} \rightarrow (\text{API/DB})$$

4.) Optimización y Rendimiento ⚡

Una excelente aplicación no solo es funcional, sino también rápida y eficiente en recursos.

4-1. Optimización de layouts

El rendimiento de la UI está directamente relacionado con la complejidad de la jerarquía de vistas (*view hierarchy*).

- **Objetivo:** Reducir la profundidad y complejidad de los anidamientos de *layouts* para minimizar el tiempo que le lleva a Android dibujar la pantalla (**Draw time**).
- **ConstraintLayout:** El *layout* más recomendado.
 - **Ventaja:** Permite crear *layouts* complejos (alineaciones, márgenes) con una estructura de vistas **plana** (poca anidación), ya que todas las vistas son hermanas del **ConstraintLayout** en lugar de ser hijas de otros *Layouts* anidados (como *LinearLayouts* dentro de *RelativeLayouts*).

4-2. Carga eficiente de imágenes

Las imágenes son una fuente común de problemas de rendimiento y consumo de memoria.

- **Problemas:** Cargar una imagen grande en la memoria puede agotar el *heap* y causar errores de **OutOfMemoryError**.
- **Glide:** La librería líder para la carga de imágenes.
 - **Funcionalidades:**
 - **Redimensión:** Escala la imagen al tamaño de la *ImageView* para no cargar innecesariamente el *bitmap* completo.
 - **Cacheo en Memoria y Disco:** Almacena copias de la imagen en la RAM y en el almacenamiento para evitar recargas.
 - **Manejo del *Lifecycle*:** Detiene la carga de la imagen si el componente (Activity/Fragment) se destruye.

4-3. Análisis de memoria

Para encontrar **fugas de memoria (*Memory Leaks*)** y problemas de rendimiento.

- **Android Profiler:** Herramienta integrada en Android Studio.
 - **Monitoriza:** Consumo de CPU, memoria, red y energía en tiempo real.
 - **Memoria:** Permite ver la asignación de memoria, *heap dumps* (instantáneas de la memoria para ver qué objetos existen) y detectar si los objetos de la *View* (como una Activity) no están siendo liberados por el recolector de basura (*Garbage Collector*) después de ser destruidos (señal de una fuga).

4-4. Pruebas de rendimiento

Medir métricas clave ayuda a cuantificar las mejoras.

- **Tiempo de Inicio (Startup Time):** El tiempo que tarda la aplicación en mostrar su primer *frame*. Google penaliza las aplicaciones con inicios lentos.
- **Fluidez (Frames Per Second - FPS):** Android apunta a 60 FPS. Una caída de FPS a menos de 60 se percibe como **"Jank"**

(saltos o *lag*).

- **Medición:** El **Android Profiler** (sección **CPU**) y el **Logcat** (buscando mensajes de *Choreographer*) pueden ayudar a identificar frames que tardan más de 16.67 ms en dibujarse ($1000 \text{ ms} / 60 \text{ frames} \approx 16.67 \text{ ms}$).
-

5.) Temas Avanzados

5-1. Notificaciones push

Permiten al servidor enviar mensajes a la aplicación Android en cualquier momento, incluso si la aplicación no está en uso.

- **Firebase Cloud Messaging (FCM):** El servicio de Google para enviar notificaciones *push*.
 - **Funcionamiento:**
 1. La aplicación se registra con FCM y recibe un **Token de Registro**.
 2. El token se envía al servidor de la aplicación.
 3. Cuando se necesita enviar una notificación, el servidor envía la carga útil (**payload**) al servidor FCM, usando el Token.
 4. FCM entrega el mensaje al dispositivo Android.
 - **Manejo en App:** Un servicio Android (`FirebaseMessagingService`) en la aplicación recibe el mensaje y decide si mostrar la notificación.

5-2. Publicación en Google Play Store

El proceso de llevar la aplicación al público.

- **Preparación y Firma:**
 - **Firma:** Toda aplicación debe estar **firmada digitalmente** con una **clave privada (keystore)**. Esta firma es crucial para que Android identifique al desarrollador y permita las actualizaciones.
 - **Bundles:** Se recomienda generar un **Android App Bundle (AAB)** en lugar de un APK tradicional. El AAB permite a Google Play optimizar la descarga para cada dispositivo, reduciendo el tamaño de la aplicación.
- **Google Play Console:** La plataforma para subir, gestionar versiones (alpha, beta, producción), metadatos (descripciones, capturas de pantalla) y monitorear el rendimiento.

5-3. Monetización

Opciones para generar ingresos con la aplicación.

- **AdMob (Anuncios):** La red de publicidad de Google. Permite integrar anuncios *banner*, intersticiales (pantalla completa) o nativos.
- **Compras dentro de la Aplicación (In-App Purchases - IAP):** Vender contenido digital (características *premium*, suscripciones, artículos virtuales). Se utiliza la **Google Play Billing Library**.

5-4. Integración de Firebase

Firebase es una plataforma que ofrece muchos servicios *backend* sin necesidad de administrar un servidor propio.

- **Analytics:** Recopila datos sobre el uso de la aplicación (cuántos usuarios la abren, qué pantallas visitan, qué botones pulsan). Es esencial para la toma de decisiones.

- **Crashlytics:** La herramienta principal para el **Reporte de Fallos (*Crash Reporting*)**.
 - **Propósito:** Cuando la aplicación falla en un dispositivo, Crashlytics registra la traza de la pila (*stack trace*) y los detalles del fallo, los agrega y los reporta en tiempo real, permitiendo al desarrollador identificar y corregir los errores críticos en producción.
-